

Savitribai Phule Pune University

S.Y.B.Sc.(C.S.) NEP – 2020 Practical Examination

Lab Course CS-203-MJ-P (Lab Course based on CS-201 MJ-T & CS-202-MJ-T)

Duration: 3 Hours

Maximum Marks: 35

Section I: Data Structures

[15 Marks]

Q 1) Write a C program to create random n elements and sort it in ascending order by using insertion sort.

OR

Q 1) Write a C program to perform the following operations using static implementation of stack

- i. Accept n integers from user and push it into the stack.
- ii. Pop the element from the stack and display it (Use isEmpty())

Section II: Database Management Systems

[15 Marks]

Q 2) Consider the following database(primary keys are underlined).

Room (roomno,room_name,room_type,charges)

Guest(Guestcode, Gname,city)

The relationship is as follows: Room-Guest: one-to-one. room_type can have values as either 'AC' or 'NonAC'.

- A. Create the above database in PostGreSQL and insert sufficient records. [5 marks]
- B. Execute the following queries in PostGreSQL (any 3) [6 marks]
 - i. List the details of the rooms having charges between 5000 and 10000.
 - ii. List the names of the guests in the sorted order by city name.
 - iii. List the minimum charges of a room.
 - iv. Increase the charges of all AC rooms by 15%.
 - v. List the names of all the NONAC rooms whose charges are more than 10000.
- C. Write a query to list the name of the guest to whom the room with highest charges is allotted. [4 marks]

OR

- C. Create a view to list the names of all the NonAC rooms which have charges greater than at least one of the 'AC' room.

Q 3) Viva

[5 marks]

SLIP 1 — Solutions (Code & Queries)

Section I: Data Structures

[15 Marks]

Q 1) Write a C program to create random n elements and sort it in ascending order by using insertion sort.

Answer / C Program

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

int main()
{
    int a[100], n, i, j, temp;

    printf("Enter number of elements: ");
    scanf("%d", &n);

    srand(time(0));

    printf("Random array: ");
    for (i = 0; i < n; i++)
    {
        a[i] = rand() % 100;
        printf("%d ", a[i]);
    }

    for (i = 1; i < n; i++)
    {
        temp = a[i];
        j = i - 1;

        while (j >= 0 && a[j] > temp)
        {
            a[j + 1] = a[j];
            j--;
        }

        a[j + 1] = temp;
    }

    printf("\nSorted array: ");
    for (i = 0; i < n; i++)
        printf("%d ", a[i]);

    return 0;
}
```

OR

Q 1) Write a C program to perform the following operations using static implementation of stack

- i. Accept n integers from user and push it into the stack.
- ii. Pop the element from the stack and display it (Use isEmpty())

Answer / C Program

```
#include <stdio.h>
#define MAX 100

int stack[MAX];
int top = -1;

int isEmpty()
{
    if (top == -1)
        return 1;
    else
        return 0;
}

void push(int x)
{
    if (top == MAX - 1)
        printf("Stack is full\n");
    else
    {
        top++;
        stack[top] = x;
    }
}

int pop()
{
    int x;
    x = stack[top];
    top--;
    return x;
}

int main()
{
    int n, i, x;

    printf("Enter number of elements: ");
    scanf("%d", &n);

    for (i = 0; i < n; i++)
    {
        printf("Enter element: ");
        scanf("%d", &x);
        push(x);
    }

    if (isEmpty())
        printf("Stack is empty\n");
    else
    {
        x = pop();
        printf("Popped element = %d\n", x);
    }
}
```

```
    return 0;
}
```

Section II: Database Management Systems

[15 Marks]

Q 2) Consider the following database(primary keys are underlined).

Room (roomno,room_name,room_type,charges)

Guest(Guestcode, Gname,city)

The relationship is as follows: Room-Guest: one-to-one. room_type can have values as either 'AC' or 'NonAC'.

A. Create the above database in PostGreSQL and insert sufficient records.

Run line-by-line in psql:

```
CREATE DATABASE slip1_db;
\c slip1_db
DROP TABLE IF EXISTS guest, room CASCADE;

CREATE TABLE room(
    roomno int PRIMARY KEY,
    room_name varchar(50),
    room_type varchar(10) CHECK (room_type IN ('AC','NonAC')),
    charges numeric(10,2)
);

CREATE TABLE guest(
    guestcode int PRIMARY KEY,
    gname varchar(50),
    city varchar(50),
    roomno int UNIQUE REFERENCES room(roomno)
);

INSERT INTO room VALUES
(101,'Deluxe','AC',8000),(102,'Suite','AC',12000),
(103,'Standard','NonAC',6000),(104,'Premium','NonAC',15000);

INSERT INTO guest VALUES
(1,'Rahul','Pune',101),(2,'Amit','Mumbai',102),
(3,'Sahil','Nashik',103),(4,'Riya','Pune',104);

SELECT * FROM room;
SELECT * FROM guest;
```

B. Execute the following queries in PostGreSQL (any 3)

i. List the details of the rooms having charges between 5000 and 10000.

```
SELECT * FROM room WHERE charges BETWEEN 5000 AND 10000;
```

ii. List the names of the guests in the sorted order by city name.

```
SELECT gname FROM guest ORDER BY city;
```

iii. List the minimum charges of a room.

```
SELECT MIN(charges) AS minimum_charges FROM room;
```

iv. Increase the charges of all AC rooms by 15%.

```
UPDATE room SET charges = charges * 1.15 WHERE room_type = 'AC';
```

v. List the names of all the NONAC rooms whose charges are more than 10000.

```
SELECT room_name FROM room WHERE room_type='NonAC' AND charges > 10000;
```

C. Write a query to list the name of the guest to whom the room with highest charges is allotted.

```
SELECT g.gname FROM guest g JOIN room r ON g.roomno=r.roomno WHERE  
r.charges=(SELECT MAX(charges) FROM room);
```

OR

C. Create a view to list the names of all the NonAC rooms which have charges greater than at least one of the 'AC' room.

```
CREATE VIEW nonac_high_rooms AS SELECT room_name, charges FROM room WHERE  
room_type='NonAC' AND charges > ANY (SELECT charges FROM room WHERE  
room_type='AC');
```

Savitribai Phule Pune University

S.Y.B.Sc.(C.S.) NEP – 2020 Practical Examination

Lab Course CS-203-MJ-P (Lab Course based on CS-201 MJ-T & CS-202-MJ-T)

Duration: 3 Hours

Maximum Marks: 35

Section I: Data Structures

[15 Marks]

Q 1) Write a C program to implement static Implementation of Queue of integers with following operation:

- Initialize()
- Insert()
- Delete()
- Isempty()
- Display()

OR

Q 1) Write a menu driven program for the following operations on singly linked list.

- i. Create and display a linked list of n nodes.
- ii. Search a particular element from a linked list.

Section II: Database Management Systems

[15 Marks]

Q 2) Consider the following database(primary keys are underlined).

Employee(eno,ename,designation,salary)

Department(dno,dname,location)

The relationship is as follows: Employee-Department: **many-to-one**.

- A. Create the above database in PostGreSQL and insert sufficient records. [5 marks]
- B. Execute the following queries in PostGreSQL (any 3) [6 marks]
 - i. Give a 5% raise in salary to all the employees.
 - ii. Display average salary of an employee.
 - iii. List the details of all the departments located at 'Nashik' city.
 - iv. Display the details of employees whose names ends with an alphabet 'a'.
 - v. Display the location of 'HR' department.
- C. Write a query to list the details of employees who do not work in any of the departments located at 'Delhi'. [4 marks]

OR

- C. Create a view to list the names of employees whose salary is greater than all the employees working in 'HR' department.

Q 3) Viva

[5 marks]

SLIP 2 — Solutions (Code & Queries)

Section I: Data Structures

[15 Marks]

Q 1) Write a C program to implement static Implementation of Queue of integers with following operation:

- Initialize()
- Insert()
- Delete()
- Isempty()
- Display()

Answer / C Program

```
#include <stdio.h>
#define MAX 100

int queue[MAX];
int front = -1, rear = -1;

void initialize()
{
    front = -1;
    rear = -1;
}

int isEmpty()
{
    if (front == -1 || front > rear)
        return 1;
    else
        return 0;
}

void insert(int x)
{
    if (rear == MAX - 1)
        printf("Queue is full\n");
    else
    {
        if (front == -1)
            front = 0;
        rear++;
        queue[rear] = x;
    }
}

void delete()
{
    if (isEmpty())
        printf("Queue is empty\n");
    else
    {
        printf("Deleted element = %d\n", queue[front]);
        front++;
    }
}

void display()
{

```

```

    int i;
    if (isEmpty())
        printf("Queue is empty\n");
    else
    {
        printf("Queue: ");
        for (i = front; i <= rear; i++)
            printf("%d ", queue[i]);
        printf("\n");
    }
}

int main()
{
    int n, i, x;

    initialize();

    printf("Enter number of elements: ");
    scanf("%d", &n);

    for (i = 0; i < n; i++)
    {
        printf("Enter element: ");
        scanf("%d", &x);
        insert(x);
    }

    display();
    delete ();
    display();

    return 0;
}

```

OR

Q 1) Write a menu driven program for the following operations on singly linked list.

- i. Create and display a linked list of n nodes.
- ii. Search a particular element from a linked list.

Answer / C Program

```

#include <stdio.h>
#include <stdlib.h>

struct node
{
    int data;
    struct node *next;
};

struct node *head = NULL;

void create(int n)
{
    struct node *newnode, *temp;
    int i, x;

    for (i = 0; i < n; i++)
    {
        newnode = (struct node *)malloc(sizeof(struct node));
    }
}

```

```

        printf("Enter data: ");
        scanf("%d", &x);

        newnode->data = x;
        newnode->next = NULL;

        if (head == NULL)
            head = newnode;
        else
        {
            temp = head;
            while (temp->next != NULL)
                temp = temp->next;

            temp->next = newnode;
        }
    }
}

void display()
{
    struct node *temp = head;

    printf("Linked list: ");
    while (temp != NULL)
    {
        printf("%d ", temp->data);
        temp = temp->next;
    }
    printf("\n");
}

void search(int x)
{
    struct node *temp = head;
    int pos = 1;

    while (temp != NULL)
    {
        if (temp->data == x)
        {
            printf("Element found at position %d\n", pos);
            return;
        }
        temp = temp->next;
        pos++;
    }

    printf("Element not found\n");
}

int main()
{
    int n, x;

    printf("Enter number of nodes: ");
    scanf("%d", &n);

    create(n);
    display();

    printf("Enter element to search: ");
    scanf("%d", &x);

```

```

    search(x);

    return 0;
}

```

Section II: Database Management Systems

[15 Marks]

Q 2) Consider the following database(primary keys are underlined).

Employee(eno,ename,designation,salary)

Department(dno,dname,location)

The relationship is as follows: Employee-Department: **many-to-one**.

A. Create the above database in PostGreSQL and insert sufficient records.

Run line-by-line in psql:

```

CREATE DATABASE slip2_db;
\c slip2_db
DROP TABLE IF EXISTS employee, department CASCADE;

CREATE TABLE department(
    dno int PRIMARY KEY,
    dname varchar(50),
    location varchar(50)
);

CREATE TABLE employee(
    eno int PRIMARY KEY,
    ename varchar(50),
    designation varchar(50),
    salary numeric(10,2),
    dno int REFERENCES department(dno)
);

INSERT INTO department VALUES
(1,'HR','Delhi'), (2,'IT','Pune'), (3,'Sales','Nashik'), (4,'Finance','Mumbai');

INSERT INTO employee VALUES
(101,'Sagar','Clerk',30000,1), (102,'Amit','Developer',50000,2),
(103,'Riya','Manager',70000,3), (104,'Pooja','HR Executive',45000,1),
(105,'Kiran','Developer',60000,2);

SELECT * FROM department;
SELECT * FROM employee;

```

B. Execute the following queries in PostGreSQL (any 3)

i. Give a 5% raise in salary to all the employees.

```
UPDATE employee SET salary = salary * 1.05;
```

ii. Display average salary of an employee.

```
SELECT AVG(salary) AS average_salary FROM employee;
```

iii. List the details of all the departments located at 'Nashik' city.

```
SELECT * FROM department WHERE location='Nashik';
```

iv. Display the details of employees whose names ends with an alphabet 'a'.

```
SELECT * FROM employee WHERE ename LIKE '%a';
```

v. Display the location of 'HR' department.

```
SELECT location FROM department WHERE dname='HR';
```

C. Write a query to list the details of employees who do not work in any of the departments located at 'Delhi'.

```
SELECT * FROM employee WHERE dno IS NULL OR dno NOT IN (SELECT dno FROM department WHERE location='Delhi');
```

OR

C. Create a view to list the names of employees whose salary is greater than all the employees working in 'HR' department.

```
CREATE VIEW high_salary_hr AS SELECT * FROM employee WHERE salary > ALL (SELECT e.salary FROM employee e JOIN department d ON e.dno=d.dno WHERE d.dname='HR');
```

Savitribai Phule Pune University

S.Y.B.Sc.(C.S.) NEP – 2020 Practical Examination

Lab Course CS-203-MJ-P (Lab Course based on CS-201 MJ-T & CS-202-MJ-T)

Duration: 3 Hours

Maximum Marks: 35

Section I: Data Structures

[15 Marks]

- Q 1) Accept the array of n integers from user and sort the array in ascending order by using recursive Quick sort algorithm.

OR

- Q 1) Create a random array of n integers. Accept a value x from user and use linear search algorithm to check whether the number is present in the array or not and output the position if the number is present.

Section II: Database Management Systems

[15 Marks]

- Q 2) Consider the following database (primary keys are underlined).

Person (pnumber, pname, birthdate, income)

Area (aname, area_type, pincode)

The relationship is as follows: Person-Area: **many-to-one**. The 'area_type' can have values as either '**urban**' or '**rural**'.

- A. Create the above database in PostgreSQL and insert sufficient records. [5 marks]
- B. Execute the following queries in PostgreSQL (any 3) [6 marks]
- List the details of all people whose name starts with the alphabet 'R'.
 - List the names of all people whose birthday falls in the month of 'July'.
 - Display the details of people in the sorted order of their income.
 - Display the count of areas of 'urban' type.
 - Change the pincode of 'kalyaninagar' to 411036.
- C. Write a query to list the names of people who live in 'Camp' area and have income less than at least one person who lives in 'kalyaninagar' area. [4 marks]

OR

- C. Create a view to list the details of the person with second maximum income.

- Q 3) Viva

[5 marks]

SLIP 3 — Solutions (Code & Queries)

Section I: Data Structures

[15 Marks]

Q 1) Accept the array of n integers from user and sort the array in ascending order by using recursive Quick sort algorithm.

Answer / C Program

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

void quicksort(int a[], int low, int high)
{
    int i, j, pivot, temp;

    if (low < high)
    {
        pivot = a[low];
        i = low;
        j = high;

        while (i < j)
        {
            while (a[i] <= pivot && i < high)
                i++;

            while (a[j] > pivot)
                j--;

            if (i < j)
            {
                temp = a[i];
                a[i] = a[j];
                a[j] = temp;
            }
        }

        a[low] = a[j];
        a[j] = pivot;

        quicksort(a, low, j - 1);
        quicksort(a, j + 1, high);
    }
}

int main()
{
    int a[100], n, i;

    printf("Enter number of elements: ");
    scanf("%d", &n);

    srand(time(0));

    printf("Random array: ");
    for (i = 0; i < n; i++)
    {
        a[i] = rand() % 100;
        printf("%d ", a[i]);
    }
}
```

```

    }

    quicksort(a, 0, n - 1);

    printf("\nSorted array: ");
    for (i = 0; i < n; i++)
        printf("%d ", a[i]);

    return 0;
}

```

OR

Q 1) Create a random array of n integers. Accept a value x from user and use linear search algorithm to check whether the number is present in the array or not and output the position if the number is present.

Answer / C Program

```

#include <stdio.h>
#include <stdlib.h>
#include <time.h>

int linearSearch(int a[], int n, int x)
{
    int i;

    for (i = 0; i < n; i++)
    {
        if (a[i] == x)
            return i + 1;
    }

    return -1;
}

int main()
{
    int a[100], n, x, i, pos;

    printf("Enter number of elements: ");
    scanf("%d", &n);

    srand(time(0));

    printf("Array: ");
    for (i = 0; i < n; i++)
    {
        a[i] = rand() % 100;
        printf("%d ", a[i]);
    }

    printf("\nEnter value to search: ");
    scanf("%d", &x);

    pos = linearSearch(a, n, x);

    if (pos == -1)
        printf("Number is not present\n");
    else
        printf("Number is present at position %d\n", pos);
}

```

```
    return 0;
}
```

Section II: Database Management Systems

[15 Marks]

Q 2) Consider the following database (primary keys are underlined).

Person (pnumber, pname, birthdate, income)

Area (aname, area_type, pincode)

The relationship is as follows: Person-Area: **many-to-one**. The 'area_type' can have values as either '**urban**' or '**rural**'.

A. Create the above database in PostGreSQL and insert sufficient records.

Run line-by-line in psql:

```
CREATE DATABASE slip3_db;
\c slip3_db
DROP TABLE IF EXISTS person, area CASCADE;

CREATE TABLE area(
    aname varchar(50) PRIMARY KEY,
    area_type varchar(10) CHECK (area_type IN ('urban','rural')),
    pincode int
);

CREATE TABLE person(
    pnumber int PRIMARY KEY,
    pname varchar(50),
    birthdate date,
    income numeric(10,2),
    aname varchar(50) REFERENCES area(aname)
);

INSERT INTO area VALUES
('Camp','urban',411001), ('kalyaninagar','urban',411014),
('Kothrud','urban',411038), ('Wagholi','rural',412207);

INSERT INTO person VALUES
(1,'Rohan','2002-07-15',18000,'Camp'),
(2,'Riya','2001-07-21',25000,'kalyaninagar'),
(3,'Raj','2000-03-10',12000,'Kothrud'),
(4,'Asha','2003-07-05',9000,'Wagholi'),
(5,'Ramesh','1999-02-12',30000,'kalyaninagar');

SELECT * FROM area;
SELECT * FROM person;
```

B. Execute the following queries in PostGreSQL (any 3)

i. List the details of all people whose name starts with the alphabet 'R'.

```
SELECT * FROM person WHERE pname LIKE 'R%';
```

ii. List the names of all people whose birthday falls in the month of 'July'.

```
SELECT pname FROM person WHERE EXTRACT(MONTH FROM birthdate)=7;
```

iii. Display the details of people in the sorted order of their income.

```
SELECT * FROM person ORDER BY income;
```

iv. Display the count of areas of 'urban' type.

```
SELECT COUNT(*) AS urban_area_count FROM area WHERE area_type='urban';
```

v. Change the pincode of 'kalyaninagar' to 411036.

```
UPDATE area SET pincode=411036 WHERE aname='kalyaninagar';
```

C. Write a query to list the names of people who live in 'Camp' area and have income less than at least one person who lives in 'kalyaninagar' area.

```
SELECT pname FROM person WHERE aname='Camp' AND income < ANY (SELECT income  
FROM person WHERE aname='kalyaninagar');
```

OR

C. Create a view to list the details of the person with second maximum income.

```
CREATE VIEW second_max_income_person AS SELECT * FROM person WHERE  
income=(SELECT MAX(income) FROM person WHERE income < (SELECT MAX(income)  
FROM person));
```

Savitribai Phule Pune University

S.Y.B.Sc.(C.S.) NEP – 2020 Practical Examination

Lab Course CS-203-MJ-P (Lab Course based on CS-201 MJ-T & CS-202-MJ-T)

Duration: 3 Hours

Maximum Marks: 35

Section I: Data Structures

[15 Marks]

Q 1) Write a C program to implement singly linked list with Create and Display operation.

OR

Q 1) Read the data from file 'student.txt' containing names of student and their class(FY,SY,TY). Accept a student name from user and use binary search algorithm to check whether the name is present in the file and output the class of student otherwise output "Student not in the list".

Section II: Database Management Systems

[15 Marks]

Q 2) Consider the following database (primary keys are underlined).

Doctor (dno, dname, city)

Patient (pno, pat_name, city, disease)

The relationship is as follows: Doctor-Patient: **many-to-many** with **Fees** as a descriptive attribute.

A. Create the above database in PostGreSQL and insert sufficient records. [5 marks]

B. Execute the following queries in PostGreSQL (any 3) [6 marks]

i. List the details of all the doctors from 'Nagpur' city.

ii. Display count of patients.

iii. List the names of the patients suffering from 'cancer' disease.

iv. Change the city of 'Dr. Patil' to Pune.

v. List the names of patients that starts with alphabet 'A'

C. Write a query to find the number of patients suffering from "Asthama" and have been charged a fee of Rs. 200 by "Dr. Kumar". [4 marks]

OR

C. Create a view to find the name of a doctor who treats maximum number of patients.

Q 3) Viva [5 marks]

SLIP 5 — Solutions (Code & Queries)

Section I: Data Structures

[15 Marks]

Q 1) Write a C program to implement singly linked list with Create and Display operation.

Answer / C Program

```
#include <stdio.h>
#include <stdlib.h>

struct node
{
    int data;
    struct node *next;
};

struct node *head = NULL;

void create(int n)
{
    struct node *newnode, *temp;
    int i, x;

    for (i = 0; i < n; i++)
    {
        newnode = (struct node *)malloc(sizeof(struct node));

        printf("Enter data: ");
        scanf("%d", &x);

        newnode->data = x;
        newnode->next = NULL;

        if (head == NULL)
            head = newnode;
        else
        {
            temp = head;
            while (temp->next != NULL)
                temp = temp->next;

            temp->next = newnode;
        }
    }
}

void display()
{
    struct node *temp = head;

    printf("Linked list: ");
    while (temp != NULL)
    {
        printf("%d ", temp->data);
        temp = temp->next;
    }
    printf("\n");
}

void search(int x)
{

```

```

    struct node *temp = head;
    int pos = 1;

    while (temp != NULL)
    {
        if (temp->data == x)
        {
            printf("Element found at position %d\n", pos);
            return;
        }
        temp = temp->next;
        pos++;
    }

    printf("Element not found\n");
}

int main()
{
    int n, x;

    printf("Enter number of nodes: ");
    scanf("%d", &n);

    create(n);
    display();

    printf("Enter element to search: ");
    scanf("%d", &x);

    search(x);

    return 0;
}

```

OR

Q 1) Read the data from file 'student.txt' containing names of student and their class(FY,SY,TY). Accept a student name from user and use binary search algorithm to check whether the name is present in the file and output the class of student otherwise output “Student not in the list”.

Answer / C Program

```

#include <stdio.h>
#include <string.h>

struct student
{
    char name[20];
    char class[5];
};

int main()
{
    FILE *fp;
    struct student s[100], temp;
    char searchName[20];
    int n = 0, i, j, low, high, mid, found = 0;

    fp = fopen("student.txt", "r");

```

```

if (fp == NULL)
{
    printf("File not found\n");
    return 0;
}

while (fscanf(fp, "%s %s", s[n].name, s[n].class) == 2)
    n++;

fclose(fp);

/* Binary search needs the names to be in sorted order. */
for (i = 0; i < n - 1; i++)
{
    for (j = i + 1; j < n; j++)
    {
        if (strcmp(s[i].name, s[j].name) > 0)
        {
            temp = s[i];
            s[i] = s[j];
            s[j] = temp;
        }
    }
}

printf("Enter student name: ");
scanf("%s", searchName);

low = 0;
high = n - 1;

while (low <= high)
{
    mid = (low + high) / 2;

    if (strcmp(s[mid].name, searchName) == 0)
    {
        printf("Student found\n");
        printf("Class = %s\n", s[mid].class);
        found = 1;
        break;
    }
    else if (strcmp(searchName, s[mid].name) < 0)
        high = mid - 1;
    else
        low = mid + 1;
}

if (found == 0)
    printf("Student not in the list\n");

return 0;
}

```

Section II: Database Management Systems

[15 Marks]

Q 2) Consider the following database (primary keys are underlined).

Person (pnumber, pname, birthdate, income)

Area (aname, area_type, pincode)

The relationship is as follows: Person-Area: **many-to-one**. The 'area_type' can have values as either '**urban**' or '**rural**'.

A. Create the above database in PostGreSQL and insert sufficient records.

Run line-by-line in psql:

```
CREATE DATABASE slip3_db;
\c slip3_db
DROP TABLE IF EXISTS person, area CASCADE;

CREATE TABLE area(
    aname varchar(50) PRIMARY KEY,
    area_type varchar(10) CHECK (area_type IN ('urban','rural')),
    pincode int
);

CREATE TABLE person(
    pnumber int PRIMARY KEY,
    pname varchar(50),
    birthdate date,
    income numeric(10,2),
    aname varchar(50) REFERENCES area(aname)
);

INSERT INTO area VALUES
('Camp','urban',411001), ('kalyaninagar','urban',411014),
('Kothrud','urban',411038), ('Wagholi','rural',412207);

INSERT INTO person VALUES
(1,'Rohan','2002-07-15',18000,'Camp'),
(2,'Riya','2001-07-21',25000,'kalyaninagar'),
(3,'Raj','2000-03-10',12000,'Kothrud'),
(4,'Asha','2003-07-05',9000,'Wagholi'),
(5,'Ramesh','1999-02-12',30000,'kalyaninagar');

SELECT * FROM area;
SELECT * FROM person;
```

B. Execute the following queries in PostGreSQL (any 3)

i. List the details of all people whose name starts with the alphabet 'R'.

```
SELECT * FROM person WHERE pname LIKE 'R%';
```

ii. List the names of all people whose birthday falls in the month of 'July'.

```
SELECT pname FROM person WHERE EXTRACT(MONTH FROM birthdate)=7;
```

iii. Display the details of people in the sorted order of their income.

```
SELECT * FROM person ORDER BY income;
```

iv. Display the count of areas of 'urban' type.

```
SELECT COUNT(*) AS urban_area_count FROM area WHERE area_type='urban';
```

v. Change the pincode of 'kalyaninagar' to 411036.

```
UPDATE area SET pincode=411036 WHERE aname='kalyaninagar';
```

C. Write a query to list the names of people who live in 'Camp' area and have income less than at least one person who lives in 'kalyaninagar' area.

```
SELECT pname FROM person WHERE aname='Camp' AND income < ANY (SELECT income  
FROM person WHERE aname='kalyaninagar');
```

OR

C. Create a view to list the details of the person with second maximum income.

```
CREATE VIEW second_max_income_person AS SELECT * FROM person WHERE  
income=(SELECT MAX(income) FROM person WHERE income < (SELECT MAX(income)  
FROM person));
```

Savitribai Phule Pune University

S.Y.B.Sc.(C.S.) NEP – 2020 Practical Examination

Lab Course CS-203-MJ-P (Lab Course based on CS-201 MJ-T & CS-202-MJ-T)

Duration: 3 Hours

Maximum Marks: 35

Section I: Data Structures

[15 Marks]

Q 1) Sort a random array of n integers (accept the value of n from user) in ascending order by using selection sort algorithm.

OR

Q 1) Write a menu driven program for the following operations on singly link list.

- i. Create and display a linked list of n nodes.
- ii. Search an element in a linked list.

Section II: Database Management Systems

[15 Marks]

Q 2) Consider the following database(primary keys are underlined).

Student (rno,name,city)

Teacher(tno,tname,phone_no,salary)

The relationship is as follows: Student-Teacher: **many-to-many** with **subject** as a descriptive attribute.

- A. Create the above database in PostGreSQL and insert sufficient records. [5 marks]
- B. Execute the following queries in PostGreSQL (any 3) [6 marks]
 - i. List the names of students from 'Ahilyanagar' city.
 - ii. Display the count of teachers whose salary is between 40000 to 60000.
 - iii. Change the phone number of 'Prof. Patil' to '9822131226'
 - iv. List the details of the teachers in the sorted order of their name.
 - v. List the names of the teachers who have salary less than 50000.
- C. Write a query to list the names of all teachers who do not teach any subject that is taught by 'Mr. Patil'. [4 marks]

OR

- C. Create a view to list the details of students who have taken 'DBMS' subject.

Q 3) Viva

[5 marks]

SLIP 6 — Solutions (Code & Queries)

Section I: Data Structures

[15 Marks]

Q 1) Sort a random array of n integers (accept the value of n from user) in ascending order by using selection sort algorithm.

Answer / C Program

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

int main()
{
    int a[100], n, i, j, min, temp;

    printf("Enter number of elements: ");
    scanf("%d", &n);

    srand(time(0));

    printf("Random array: ");
    for (i = 0; i < n; i++)
    {
        a[i] = rand() % 100;
        printf("%d ", a[i]);
    }

    for (i = 0; i < n - 1; i++)
    {
        min = i;

        for (j = i + 1; j < n; j++)
        {
            if (a[j] < a[min])
                min = j;
        }

        temp = a[i];
        a[i] = a[min];
        a[min] = temp;
    }

    printf("\nSorted array: ");
    for (i = 0; i < n; i++)
        printf("%d ", a[i]);

    return 0;
}
```

OR

Q 1) Write a menu driven program for the following operations on singly link list.

- i. Create and display a linked list of n nodes.
- ii. Search an element in a linked list.

Answer / C Program

```
#include <stdio.h>
#include <stdlib.h>

struct node
{
    int data;
    struct node *next;
};

struct node *head = NULL;

void create(int n)
{
    struct node *newnode, *temp;
    int i, x;

    for (i = 0; i < n; i++)
    {
        newnode = (struct node *)malloc(sizeof(struct node));

        printf("Enter data: ");
        scanf("%d", &x);

        newnode->data = x;
        newnode->next = NULL;

        if (head == NULL)
            head = newnode;
        else
        {
            temp = head;
            while (temp->next != NULL)
                temp = temp->next;
            temp->next = newnode;
        }
    }
}

void display()
{
    struct node *temp = head;

    printf("Linked list: ");
    while (temp != NULL)
    {
        printf("%d ", temp->data);
        temp = temp->next;
    }
    printf("\n");
}

void search(int x)
{
    struct node *temp = head;
```

```

while (temp != NULL)
{
    if (temp->data == x)
    {
        printf("Element found\n");
        return;
    }
    temp = temp->next;
}

printf("Element not found\n");
}

int main()
{
    int n, x;

    printf("Enter number of nodes: ");
    scanf("%d", &n);

    create(n);
    display();

    printf("Enter element to search: ");
    scanf("%d", &x);

    search(x);

    return 0;
}

```

Section II: Database Management Systems

[15 Marks]

Q 2) Consider the following database(primary keys are underlined).

Student (rno,name,city)

Teacher(tno,tname,phone_no,salary)

The relationship is as follows: Student-Teacher: **many-to-many** with **subject** as a descriptive attribute.

A. Create the above database in PostGreSQL and insert sufficient records.

Run line-by-line in psql:

```

CREATE DATABASE slip6_db;
\c slip6_db
DROP TABLE IF EXISTS student_teacher, student, teacher CASCADE;

CREATE TABLE student(rno int PRIMARY KEY,name varchar(50),city varchar(50));
CREATE TABLE teacher(tno int PRIMARY KEY,tname varchar(50),phone_no
varchar(15),salary numeric(10,2));

CREATE TABLE student_teacher(
    rno int REFERENCES student(rno),
    tno int REFERENCES teacher(tno),
    subject varchar(50),
    PRIMARY KEY(rno,tno,subject)
);

INSERT INTO student VALUES
(1,'Amit','Ahilyanagar'), (2,'Riya','Pune'), (3,'Sahil','Ahilyanagar'), (4,'Pooja','Nashik');

```

```

INSERT INTO teacher VALUES
(101,'Mr. Patil','9822131226',55000),(102,'Mrs. Shah','9822000000',45000),
(103,'Mr. Joshi','9822111111',65000),(104,'Mr. Wani','9822222222',48000);

INSERT INTO student_teacher VALUES
(1,101,'DBMS'),(1,102,'Maths'),(2,101,'OS'),
(2,103,'DBMS'),(3,102,'Maths'),(3,104,'C'),(4,101,'DBMS');

SELECT * FROM student;
SELECT * FROM teacher;
SELECT * FROM student_teacher;

```

B. Execute the following queries in PostgreSQL (any 3)

i. List the names of students from ‘Ahilyanagar’ city.

```
SELECT name FROM student WHERE city='Ahilyanagar';
```

ii. Display the count of teachers whose salary is between 40000 to 60000.

```
SELECT COUNT(*) FROM teacher WHERE salary BETWEEN 40000 AND 60000;
```

iii. Change the phone number of ‘Prof. Patil’ to ‘9822131226’

```
UPDATE teacher SET phone_no='9822131226' WHERE tname='Prof. Patil';
```

iv. List the details of the teachers in the sorted order of their name.

```
SELECT * FROM teacher ORDER BY tname;
```

v. List the names of the teachers who have salary less than 50000.

```
SELECT tname FROM teacher WHERE salary < 50000;
```

C. Write a query to list the names of all teachers who do not teach any subject that is taught by ‘Mr. Patil’.

```

SELECT DISTINCT t.tname FROM teacher t WHERE NOT EXISTS (SELECT 1 FROM
student_teacher st WHERE st.tno=t.tno AND st.subject IN (SELECT subject FROM
student_teacher WHERE tno=(SELECT tno FROM teacher WHERE tname='Mr.
Patil')));

```

OR

C. Create a view to list the details of students who have taken ‘DBMS’ subject.

```

CREATE VIEW dbms_students AS SELECT s.* FROM student s JOIN student_teacher
st ON s.rno=st.rno WHERE st.subject='DBMS';

```

Savitribai Phule Pune University

S.Y.B.Sc.(C.S.) NEP – 2020 Practical Examination

Lab Course CS-203-MJ-P (Lab Course based on CS-201 MJ-T & CS-202-MJ-T)

Duration: 3 Hours

Maximum Marks: 35

Section I: Data Structures

[15 Marks]

Q 1) Sort a random array of n integers (accept the value of n from user) in ascending order by using bubble sort algorithm.

OR

Q 1) Write a menu driven program for the following operations on doubly link list.

- i. Create and Display a link list of n nodes
- ii. Insert an element in a linked list in a particular position

Section II: Database Management Systems

[15 Marks]

Q 2) Consider the following Student-Teacher database(primary keys are underlined).

Student (sno , s_name, s_class)

Teacher (tno , t_name, yrs_experience)

The relationship is as follows:

Student-Teacher: **Many to Many** with descriptive attribute **Subject**.

Constraint:s_class should be either "FY", "SY" or "TY"

A. Create the above database in PostGreSQL and insert sufficient records. [5 marks]

B. Execute the following queries in PostGreSQL (any 3) [6 marks]

- i. Give class-wise number of students.
- ii. List all students studying in class "TY".
- iii. Count the number of students who have taken subject "Maths"
- iv. Delete record of student whose sno "105"
- v. Add designation column in teacher table

C. Write a query to list the names of all teachers with their subjects along with the total number of students they are teaching. [4 marks]

OR

C. Create a view to find the teacher who teaches maximum number of subjects.

Q 3) Viva [5 marks]

SLIP 8 — Solutions (Code & Queries)

Section I: Data Structures

[15 Marks]

Q 1) Sort a random array of n integers (accept the value of n from user) in ascending order by using bubble sort algorithm.

Answer / C Program

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

int main()
{
    int a[100], n, i, j, temp;

    printf("Enter number of elements: ");
    scanf("%d", &n);

    srand(time(0));

    printf("Random array: ");
    for (i = 0; i < n; i++)
    {
        a[i] = rand() % 100;
        printf("%d ", a[i]);
    }

    for (i = 0; i < n - 1; i++)
    {
        for (j = 0; j < n - i - 1; j++)
        {
            if (a[j] > a[j + 1])
            {
                temp = a[j];
                a[j] = a[j + 1];
                a[j + 1] = temp;
            }
        }
    }

    printf("\nSorted array: ");
    for (i = 0; i < n; i++)
        printf("%d ", a[i]);

    return 0;
}
```

OR

Q 1) Write a menu driven program for the following operations on doubly link list.

- i. Create and Display a link list of n nodes
- ii. Insert an element in a linked list in a particular position

Answer / C Program

```
#include <stdio.h>
#include <stdlib.h>

struct node
{
    int data;
    struct node *prev;
    struct node *next;
};

struct node *head = NULL;

void create(int n)
{
    struct node *newnode, *temp;
    int i, x;

    for (i = 0; i < n; i++)
    {
        newnode = (struct node *)malloc(sizeof(struct node));

        printf("Enter data: ");
        scanf("%d", &x);

        newnode->data = x;
        newnode->prev = NULL;
        newnode->next = NULL;

        if (head == NULL)
            head = newnode;
        else
        {
            temp = head;
            while (temp->next != NULL)
                temp = temp->next;

            temp->next = newnode;
            newnode->prev = temp;
        }
    }
}

void display()
{
    struct node *temp = head;

    printf("Doubly linked list: ");
    while (temp != NULL)
    {
        printf("%d ", temp->data);
        temp = temp->next;
    }
    printf("\n");
}
```

```

void insertPosition(int x, int pos)
{
    struct node *newnode, *temp;
    int i;

    newnode = (struct node *)malloc(sizeof(struct node));
    newnode->data = x;
    newnode->prev = NULL;
    newnode->next = NULL;

    if (pos == 1)
    {
        newnode->next = head;
        if (head != NULL)
            head->prev = newnode;
        head = newnode;
        return;
    }

    temp = head;

    for (i = 1; i < pos - 1 && temp != NULL; i++)
        temp = temp->next;

    if (temp == NULL)
    {
        printf("Invalid position\n");
        free(newnode);
        return;
    }

    newnode->next = temp->next;
    newnode->prev = temp;

    if (temp->next != NULL)
        temp->next->prev = newnode;

    temp->next = newnode;
}

int main()
{
    int n, x, pos;

    printf("Enter number of nodes: ");
    scanf("%d", &n);

    create(n);
    display();

    printf("Enter element to insert: ");
    scanf("%d", &x);

    printf("Enter position: ");
    scanf("%d", &pos);

    insertPosition(x, pos);
    display();

    return 0;
}

```

Section II: Database Management Systems

[15 Marks]

Q 2) Consider the following Student-Teacher database(primary keys are underlined).

Student (sno , s_name, s_class)

Teacher (tno , t_name, yrs_experience)

The relationship is as follows:

Student-Teacher: **Many to Many** with descriptive attribute **Subject**.

Constraint:s_class should be either "FY", "SY" or "TY"

A. Create the above database in PostGreSQL and insert sufficient records.

Run line-by-line in psql:

```
CREATE DATABASE slip8_db;
\c slip8_db
DROP TABLE IF EXISTS student_teacher, student, teacher CASCADE;

CREATE TABLE student(
    sno int PRIMARY KEY,
    s_name varchar(50),
    s_class varchar(2) CHECK (s_class IN ('FY','SY','TY'))
);

CREATE TABLE teacher(tno int PRIMARY KEY,t_name varchar(50),yrs_experience
int);

CREATE TABLE student_teacher(
    sno int REFERENCES student(sno),
    tno int REFERENCES teacher(tno),
    subject varchar(50),
    PRIMARY KEY(sno,tno,subject)
);

INSERT INTO student VALUES
(101,'Amit','FY'), (102,'Riya','SY'), (103,'Sahil','TY'), (104,'Pooja','TY'), (10
5,'Kiran','FY');

INSERT INTO teacher VALUES
(1,'Patil',8), (2,'Shah',10), (3,'Joshi',5);

INSERT INTO student_teacher VALUES
(101,1,'Maths'), (102,1,'DBMS'), (103,1,'Maths'),
(103,2,'DBMS'), (104,2,'Maths'), (105,3,'C');

SELECT * FROM student;
SELECT * FROM teacher;
SELECT * FROM student_teacher;
```

B. Execute the following queries in PostGreSQL (any 3)

i. Give class-wise number of students.

```
SELECT s_class, COUNT(*) AS student_count FROM student GROUP BY s_class;
```

ii. List all students studying in class "TY".

```
SELECT * FROM student WHERE s_class='TY';
```

iii. Count the number of students who have taken subject “Maths”

```
SELECT COUNT(DISTINCT sno) FROM student_teacher WHERE subject='Maths';
```

iv. Delete record of student whose sno “105”

```
DELETE FROM student WHERE sno=105;
```

v. Add designation column in teacher table

```
ALTER TABLE teacher ADD COLUMN designation varchar(50);
```

C. Write a query to list the names of all teachers with their subjects along with the total number of students they are teaching.

```
SELECT t.t_name, st.subject, COUNT(DISTINCT st.sno) AS total_students FROM  
teacher t JOIN student_teacher st ON t.tno=st.tno GROUP BY  
t.t_name,st.subject ORDER BY t.t_name,st.subject;
```

OR

C. Create a view to find the teacher who teaches maximum number of subjects.

```
CREATE VIEW max_subject_teacher AS SELECT t.t_name, COUNT(DISTINCT  
st.subject) AS subject_count FROM teacher t JOIN student_teacher st ON  
t.tno=st.tno GROUP BY t.tno,t.t_name ORDER BY subject_count DESC LIMIT 1;
```

Savitribai Phule Pune University

S.Y.B.Sc.(C.S.) NEP – 2020 Practical Examination

Lab Course CS-203-MJ-P (Lab Course based on CS-201 MJ-T & CS-202-MJ-T)

Duration: 3 Hours

Maximum Marks: 35

Section I: Data Structures

[15 Marks]

Q 1) Write a menu driven program for the following operations on singly linked list.

- i. Create and display a linked list of n nodes.
- ii. Delete a particular element from a linked list.

OR

Q 1) Create a random array of n integers. Accept a value x from user and use linear search algorithm to check whether the number is present in the array or not and output the position if the number is present.

- i) Accept & Display Function
- ii) Linear Search Function

Section II: Database Management Systems

[15 Marks]

Q 2) Create the following tables (primary keys are underlined).

Person (pnumber, pname, birthdate, income),

Area(aname,area_type).

There exists a **one-to-many** relationship between Area and Person.

Constraint: 'area_type' should have value either **urban** or **rural**.

A. Create the above database in PostGreSQL and insert sufficient records. [5 marks]

B. Execute the following queries in PostGreSQL (any 3) [6 marks]

- i. List the details of persons with income greater than 10000.
- ii. Transfer all people living in 'Pune' to 'Delhi'.
- iii. Count number of area's which has type 'urban'.
- iv. Give the count of people who are born in the month of February.
- v. List the names of all people living in 'rural' area.

C. Write a query that count the total number of people staying in "urban" area with income equal to average income. [4 marks]

OR

C. Create a view to print the person's details along with their area name.

Q 3) Viva [5 marks]

SLIP 9 — Solutions (Code & Queries)

Section I: Data Structures

[15 Marks]

Q 1) Write a menu driven program for the following operations on singly linked list.

- i. Create and display a linked list of n nodes.
- ii. Delete a particular element from a linked list.

Answer / C Program

```
#include <stdio.h>
#include <stdlib.h>

struct node
{
    int data;
    struct node *next;
};

struct node *head = NULL;

void create(int n)
{
    struct node *newnode, *temp;
    int i, x;

    for (i = 0; i < n; i++)
    {
        newnode = (struct node *)malloc(sizeof(struct node));

        printf("Enter data: ");
        scanf("%d", &x);

        newnode->data = x;
        newnode->next = NULL;

        if (head == NULL)
            head = newnode;
        else
        {
            temp = head;
            while (temp->next != NULL)
                temp = temp->next;
            temp->next = newnode;
        }
    }
}

void display()
{
    struct node *temp = head;

    printf("Linked list: ");
    while (temp != NULL)
    {
        printf("%d ", temp->data);
        temp = temp->next;
    }
    printf("\n");
}
```

```

void deleteElement(int x)
{
    struct node *temp = head, *prev = NULL;

    while (temp != NULL && temp->data != x)
    {
        prev = temp;
        temp = temp->next;
    }

    if (temp == NULL)
    {
        printf("Element not found\n");
        return;
    }

    if (prev == NULL)
        head = temp->next;
    else
        prev->next = temp->next;

    free(temp);
    printf("Element deleted\n");
}

int main()
{
    int n, x;

    printf("Enter number of nodes: ");
    scanf("%d", &n);

    create(n);
    display();

    printf("Enter element to delete: ");
    scanf("%d", &x);

    deleteElement(x);
    display();

    return 0;
}

```

OR

Q 1) Create a random array of n integers. Accept a value x from user and use linear search algorithm to check whether the number is present in the array or not and output the position if the number is present.

- i) Accept & Display Function
- ii) Linear Search Function

Answer / C Program

```

#include <stdio.h>
#include <stdlib.h>
#include <time.h>

void accept(int a[], int n)
{
    int i;

```

```

    srand(time(0));

    for (i = 0; i < n; i++)
        a[i] = rand() % 100;
}

void display(int a[], int n)
{
    int i;

    printf("Array: ");
    for (i = 0; i < n; i++)
        printf("%d ", a[i]);
    printf("\n");
}

int linearSearch(int a[], int n, int x)
{
    int i;

    for (i = 0; i < n; i++)
    {
        if (a[i] == x)
            return i + 1;
    }

    return -1;
}

int main()
{
    int a[100], n, x, pos;

    printf("Enter number of elements: ");
    scanf("%d", &n);

    accept(a, n);
    display(a, n);

    printf("Enter value to search: ");
    scanf("%d", &x);

    pos = linearSearch(a, n, x);

    if (pos == -1)
        printf("Number is not present\n");
    else
        printf("Number is present at position %d\n", pos);

    return 0;
}

```

Section II: Database Management Systems

[15 Marks]

Q 2) Create the following tables (primary keys are underlined).

Person (pnumber, pname, birthdate, income),

Area(aname,area_type).

There exists a **one-to-many** relationship between Area and Person.

Constraint: 'area_type' should have value either **urban** or **rural**.

A. Create the above database in PostGreSQL and insert sufficient records.

Run line-by-line in psql:

```
CREATE DATABASE slip9_db;
\c slip9_db
DROP TABLE IF EXISTS person, area CASCADE;

CREATE TABLE area(
    aname varchar(50) PRIMARY KEY,
    area_type varchar(10) CHECK (area_type IN ('urban','rural'))
);

CREATE TABLE person(
    pnumber int PRIMARY KEY,
    pname varchar(50),
    birthdate date,
    income numeric(10,2),
    aname varchar(50) REFERENCES area(aname)
);

INSERT INTO area VALUES
('Pune','urban'),('Delhi','urban'),('Camp','urban'),('Wagholi','rural');

INSERT INTO person VALUES
(1,'Amit','2000-02-10',10000,'Pune'),
(2,'Riya','2001-02-15',20000,'Delhi'),
(3,'Sahil','2002-03-20',15000,'Camp'),
(4,'Pooja','2003-02-25',12000,'Wagholi'),
(5,'Rohan','2000-05-10',18000,'Pune');

SELECT * FROM area;
SELECT * FROM person;
```

B. Execute the following queries in PostGreSQL (any 3)

i. List the details of persons with income greater than 10000.

```
SELECT * FROM person WHERE income > 10000;
```

ii. Transfer all people living in 'Pune' to 'Delhi'.

```
UPDATE person SET aname='Delhi' WHERE aname='Pune';
```

iii. Count number of area's which has type 'urban'.

```
SELECT COUNT(*) FROM area WHERE area_type='urban';
```

iv. Give the count of people who are born in the month of February.

```
SELECT COUNT(*) FROM person WHERE EXTRACT(MONTH FROM birthdate)=2;
```

v. List the names of all people living in 'rural' area.

```
SELECT p.pname FROM person p JOIN area a ON p.aname=a.aname WHERE  
a.area_type='rural';
```

C. Write a query that count the total number of people staying in “urban” area with income equal to average income.

```
SELECT COUNT(*) FROM person p JOIN area a ON p.aname=a.aname WHERE  
a.area_type='urban' AND p.income=(SELECT AVG(income) FROM person);
```

OR

C. Create a view to print the person’s details along with their area name.

```
CREATE VIEW person_area_details AS SELECT  
p.pnumber,p.pname,p.birthdate,p.income,a.aname,a.area_type FROM person p JOIN  
area a ON p.aname=a.aname;
```

Savitribai Phule Pune University

S.Y.B.Sc.(C.S.) NEP – 2020 Practical Examination

Lab Course CS-203-MJ-P (Lab Course based on CS-201 MJ-T & CS-202-MJ-T)

Duration: 3 Hours

Maximum Marks: 35

Section I: Data Structures

[15 Marks]

Q 1) Create random array of n integers and sort the array in ascending order by using recursive Merge sort algorithm

OR

Q 1) Write a C program to implement Static implementation of stack of integers with following operation.

- a. Initialize()
- b. Push()
- c. Pop()
- d. Isempy()
- e. Isfull()
- f. Peek()

Section II: Database Management Systems

[15 Marks]

Q 2) Create the following tables (primary keys are underlined).

Account (acctno, acct_type, balance, branch_name)

Customer (custno, cust_name, cust_city)

Relationships: Customer-Account: **One to Many.**

Constraints: acct_type should be “**saving**” or “**current**” and balance is greater than zero.

- A. Create the above database in PostGreSQL and insert sufficient records. [5 marks]
- B. Execute the following queries in PostGreSQL (any 3) [6 marks]
 - i. Display information of all saving accounts having balance greater than 5000.
 - ii. Count customers in city “Pune”.
 - iii. Find the total balance at branch “M.G.Road”.
 - iv. Delete the record whose cust_name is “Ramnath”.
 - v. Change city of customer “Sanjay” to “Pune”

- C. Write a query to find the number of account holders for each city. [4 marks]

OR

- C. Create a view to print the account details along with customer and their branch details.

Q 3) Viva

[5 marks]

SLIP 10 — Solutions (Code & Queries)

Section I: Data Structures

[15 Marks]

Q 1) Create random array of n integers and sort the array in ascending order by using recursive Merge sort algorithm

Answer / C Program

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

void merge(int a[], int low, int mid, int high)
{
    int b[100], i, j, k;

    i = low;
    j = mid + 1;
    k = low;

    while (i <= mid && j <= high)
    {
        if (a[i] < a[j])
            b[k++] = a[i++];
        else
            b[k++] = a[j++];
    }

    while (i <= mid)
        b[k++] = a[i++];

    while (j <= high)
        b[k++] = a[j++];

    for (i = low; i <= high; i++)
        a[i] = b[i];
}

void mergeSort(int a[], int low, int high)
{
    int mid;

    if (low < high)
    {
        mid = (low + high) / 2;

        mergeSort(a, low, mid);
        mergeSort(a, mid + 1, high);

        merge(a, low, mid, high);
    }
}

int main()
{
    int a[100], n, i;

    printf("Enter number of elements: ");
    scanf("%d", &n);
```

```

    srand(time(0));

    printf("Random array: ");
    for (i = 0; i < n; i++)
    {
        a[i] = rand() % 100;
        printf("%d ", a[i]);
    }

    mergeSort(a, 0, n - 1);

    printf("\nSorted array: ");
    for (i = 0; i < n; i++)
        printf("%d ", a[i]);

    return 0;
}

```

OR

Q 1) Write a C program to implement Static implementation of stack of integers with following operation.

- a. Initialize()
- b. Push()
- c. Pop()
- d. Isempty()
- e. Isfull()
- f. Peek()

Answer / C Program

```

#include <stdio.h>
#define MAX 100

int stack[MAX];
int top;

void Initialize()
{
    top = -1;
}

int IsEmpty()
{
    if (top == -1)
        return 1;
    else
        return 0;
}

int IsFull()
{
    if (top == MAX - 1)
        return 1;
    else
        return 0;
}

void Push(int x)
{

```

```

    if (IsFull())
        printf("Stack is full\n");
    else
    {
        top++;
        stack[top] = x;
    }
}

void Pop()
{
    if (IsEmpty())
        printf("Stack is empty\n");
    else
    {
        printf("Popped element = %d\n", stack[top]);
        top--;
    }
}

void Peek()
{
    if (IsEmpty())
        printf("Stack is empty\n");
    else
        printf("Top element = %d\n", stack[top]);
}

int main()
{
    int n, i, x;

    Initialize();

    printf("Enter number of elements: ");
    scanf("%d", &n);

    for (i = 0; i < n; i++)
    {
        printf("Enter element: ");
        scanf("%d", &x);
        Push(x);
    }

    Peek();
    Pop();
    Peek();

    return 0;
}

```

Section II: Database Management Systems

[15 Marks]

Q 2) Create the following tables (primary keys are underlined).

Account (acctno ,acct_type, balance, branch_name)

Customer (custno, cust_name, cust_city)

Relationships: Customer-Account: **One to Many**.

Constraints: acct_type should be “**saving**” or “**current**”and **balance is greater than zero**.

A. Create the above database in PostGreSQL and insert sufficient records.

Run line-by-line in psql:

```
CREATE DATABASE slip10_db;
\c slip10_db
DROP TABLE IF EXISTS account, customer CASCADE;

CREATE TABLE customer(
    custno int PRIMARY KEY,
    cust_name varchar(50),
    cust_city varchar(50)
);

CREATE TABLE account(
    acctno int PRIMARY KEY,
    acct_type varchar(10) CHECK (acct_type IN ('saving','current')),
    balance numeric(12,2) CHECK (balance > 0),
    branch_name varchar(50),
    custno int REFERENCES customer(custno)
);

INSERT INTO customer VALUES
(1, 'Ramnath', 'Pune'), (2, 'Sanjay', 'Pune'), (3, 'Amit', 'Mumbai'), (4, 'Riya', 'Nashik');

INSERT INTO account VALUES
(101, 'saving', 8000, 'M.G.Road', 1), (102, 'current', 12000, 'M.G.Road', 1),
(103, 'saving', 4000, 'Camp', 2), (104, 'saving', 9000, 'Camp', 3),
(105, 'current', 15000, 'M.G.Road', 4);

SELECT * FROM customer;
SELECT * FROM account;
```

B. Execute the following queries in PostGreSQL (any 3)

i. Display information of all saving accounts having balance greater than 5000.

```
SELECT * FROM account WHERE acct_type='saving' AND balance>5000;
```

ii. Count customers in city “Pune”.

```
SELECT COUNT(*) FROM customer WHERE cust_city='Pune';
```

iii. Find the total balance at branch “M.G.Road”.

```
SELECT SUM(balance) AS total_balance FROM account WHERE
branch_name='M.G.Road';
```

iv. Delete the record whose cust_name is “Ramnath”.

```
DELETE FROM customer WHERE cust_name='Ramnath';
```

v. Change city of customer “Sanjay” to “Pune”

```
UPDATE customer SET cust_city='Pune' WHERE cust_name='Sanjay';
```

C. Write a query to find the number of account holders for each city.

```
SELECT c.cust_city, COUNT(DISTINCT c.custno) AS account_holders FROM customer  
c JOIN account a ON c.custno=a.custno GROUP BY c.cust_city;
```

OR

C. Create a view to print the account details along with customer and their branch details.

```
CREATE VIEW account_customer_branch AS SELECT  
a.acctno,a.acct_type,a.balance,a.branch_name,c.cust_name,c.cust_city FROM  
account a JOIN customer c ON a.custno=c.custno;
```

Savitribai Phule Pune University

S.Y.B.Sc.(C.S.) NEP – 2020 Practical Examination

Lab Course CS-203-MJ-P (Lab Course based on CS-201 MJ-T & CS-202-MJ-T)

Duration: 3 Hours

Maximum Marks: 35

Section I: Data Structures

[15 Marks]

- Q 1) Perform the following operations using dynamic implementation of stack
- Accept n integers from user and push it into the stack.
 - Reverses a string of characters using stack and check whether given string is palindrome or not.

OR

- Q 1) Sort a random array of n integers (accept the value of n from user) in ascending order by using selection sort algorithm.

Section II: Database Management Systems

[15 Marks]

- Q 2) Create the following tables (**primary keys are underlined**).

Sales_order(sorderno, s_order_date, order_amt)

Client(clientno, name, address)

There exists a **one-to-many** relationship between Client and Sales_order

Constraints: order_amt should be greater than zero.

- Create the above database in PostgreSQL and insert sufficient records. [5 marks]
- Execute the following queries in PostgreSQL (any 3) [6 marks]
 - Update the client address of all clients from “Nagpur” to “Kolhapur”.
 - Delete all sales order whose client number is 30;
 - Display all sale records having order date before “23/02/2024”.
 - Display sales order having maximum order amount.
 - Add column order_status to the Sales_order table.

- Write a query to Display client details having maximum sales orders. [4 marks]

OR

- Create a view to list names of clients having more than 2 sales orders.

- Q 3) Viva

[5 marks]

SLIP 12 — Solutions (Code & Queries)

Section I: Data Structures

[15 Marks]

Q 1) Perform the following operations using dynamic implementation of stack

- Accept n integers from user and push it into the stack.
- Reverses a string of characters using stack and check whether given string is palindrome or not.

Answer / C Program

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

struct intnode
{
    int data;
    struct intnode *next;
};

struct intnode *top = NULL;

void push(int x)
{
    struct intnode *newnode;

    newnode = (struct intnode *)malloc(sizeof(struct intnode));
    newnode->data = x;
    newnode->next = top;
    top = newnode;
}

void display()
{
    struct intnode *temp = top;

    printf("Stack: ");
    while (temp != NULL)
    {
        printf("%d ", temp->data);
        temp = temp->next;
    }
    printf("\n");
}

void reverseString(char str[])
{
    char rev[100];
    char stack[100];
    int top = -1, i, n;

    n = strlen(str);

    for (i = 0; i < n; i++)
        stack[++top] = str[i];

    for (i = 0; i < n; i++)
        rev[i] = stack[top--];
}
```

```

    rev[n] = '\0';

    printf("Reversed string = %s\n", rev);

    if (strcmp(str, rev) == 0)
        printf("String is palindrome\n");
    else
        printf("String is not palindrome\n");
}

int main()
{
    int n, i, x;
    char str[100];

    printf("Enter number of integers: ");
    scanf("%d", &n);

    for (i = 0; i < n; i++)
    {
        printf("Enter integer: ");
        scanf("%d", &x);
        push(x);
    }

    display();

    printf("Enter string: ");
    scanf("%s", str);

    reverseString(str);

    return 0;
}

```

OR

Q 1) Sort a random array of n integers (accept the value of n from user) in ascending order by using selection sort algorithm.

Answer / C Program

```

#include <stdio.h>
#include <stdlib.h>
#include <time.h>

int main()
{
    int a[100], n, i, j, min, temp;

    printf("Enter number of elements: ");
    scanf("%d", &n);

    srand(time(0));

    printf("Random array: ");
    for (i = 0; i < n; i++)
    {
        a[i] = rand() % 100;
        printf("%d ", a[i]);
    }
}

```

```

for (i = 0; i < n - 1; i++)
{
    min = i;

    for (j = i + 1; j < n; j++)
    {
        if (a[j] < a[min])
            min = j;
    }

    temp = a[i];
    a[i] = a[min];
    a[min] = temp;
}

printf("\nSorted array: ");
for (i = 0; i < n; i++)
    printf("%d ", a[i]);

return 0;
}

```

Section II: Database Management Systems

[15 Marks]

Q 2) Create the following tables (primary keys are underlined).

Sales_order(sorderno, s_order_date, order_amt)

Client(clientno, name, address)

There exists a **one-to-many** relationship between Client and Sales_order

Constraints: order_amt should be greater than zero.

A. Create the above database in PostGreSQL and insert sufficient records.

Run line-by-line in psql:

```

CREATE DATABASE slip12_db;
\c slip12_db
DROP TABLE IF EXISTS sales_order, client CASCADE;

CREATE TABLE client(
    clientno int PRIMARY KEY,
    name varchar(50),
    address varchar(50)
);

CREATE TABLE sales_order(
    sorderno int PRIMARY KEY,
    s_order_date date,
    order_amt numeric(12,2) CHECK (order_amt > 0),
    clientno int REFERENCES client(clientno)
);

INSERT INTO client VALUES
(10, 'Amit', 'Pune'), (20, 'Riya', 'Nagpur'), (30, 'Sahil', 'Nagpur'), (40, 'Pooja', 'Mumbai');

INSERT INTO sales_order VALUES
(1, '2024-01-10', 25000, 10), (2, '2024-02-20', 45000, 10),
(3, '2024-03-05', 15000, 20), (4, '2024-10-12', 60000, 30), (5, '2024-10-20', 35000, 30);

SELECT * FROM client;

```

```
SELECT * FROM sales_order;
```

B. Execute the following queries in PostGreSQL (any 3)

i. Update the client address of all clients from “Nagpur” to “Kolhapur”.

```
UPDATE client SET address='Kolhapur' WHERE address='Nagpur';
```

ii. Delete all sales order whose client number is 30;

```
DELETE FROM sales_order WHERE clientno=30;
```

iii. Display all sale records having order date before “23/02/2024”.

```
SELECT * FROM sales_order WHERE s_order_date < DATE '2024-02-23';
```

iv. Display sales order having maximum order amount.

```
SELECT * FROM sales_order WHERE order_amt=(SELECT MAX(order_amt) FROM sales_order);
```

v. Add column order_status to the Sales_order table.

```
ALTER TABLE sales_order ADD COLUMN order_status varchar(30);
```

C. Write a query to Display client details having maximum sales orders.

```
SELECT c.clientno,c.name,c.address,COUNT(s.sorderno) AS order_count FROM client c JOIN sales_order s ON c.clientno=s.clientno GROUP BY c.clientno,c.name,c.address ORDER BY order_count DESC LIMIT 1;
```

OR

C. Create a view to list names of clients having more than 2 sales orders.

```
CREATE VIEW clients_more_than_2_orders AS SELECT c.clientno,c.name,COUNT(s.sorderno) AS order_count FROM client c JOIN sales_order s ON c.clientno=s.clientno GROUP BY c.clientno,c.name HAVING COUNT(s.sorderno)>2;
```

Savitribai Phule Pune University

S.Y.B.Sc.(C.S.) NEP – 2020 Practical Examination

Lab Course CS-203-MJ-P (Lab Course based on CS-201 MJ-T & CS-202-MJ-T)

Duration: 3 Hours

Maximum Marks: 35

Section I: Data Structures

[15 Marks]

- Q 1) Accept n sorted values in array from user. Accept a value x from user and use binary search algorithm to check whether the number is present in sorted array or not and output the position if the number is present.

OR

- Q 1) Perform the following operations using dynamic implementation of Queue
- Insert n elements in a queue.
 - Delete the element from the queue and display it.

Section II: Database Management Systems

[15 Marks]

- Q 2) Create the following tables (**primary keys are underlined**).

Bus (Busno , capacity ,depot_name)

Route (Routeno ,source ,destination ,no_of_stations)

Relationship: Bus-Route : **Many to one**

Constraint: Bus capacity should not be null.

- Create the above database in PostGreSQL and insert sufficient records. [5 marks]
- Execute the following queries in PostGreSQL (any 3) [6 marks]
 - List all buses at depot “Swargate”.
 - Delete Bus whose Bus number is “204”.
 - List all buses on route number 41.
 - List the route details having number of stations greater than 10.
 - List all routes starting from “Pune station”.

- Write a query to delete all the buses on routes in which total stations are less than 3. [4 marks]

OR

- Create a view to list all buses and route details from “Pimpri” to “Hadapsar”.

- Q 3) Viva

[5 marks]

SLIP 13 — Solutions (Code & Queries)

Section I: Data Structures

[15 Marks]

Q 1) Accept n sorted values in array from user. Accept a value x from user and use binary search algorithm to check whether the number is present in sorted array or not and output the position if the number is present.

Answer / C Program

```
#include <stdio.h>

int binarySearch(int a[], int n, int x)
{
    int low = 0, high = n - 1, mid;

    while (low <= high)
    {
        mid = (low + high) / 2;

        if (a[mid] == x)
            return mid + 1;
        else if (x < a[mid])
            high = mid - 1;
        else
            low = mid + 1;
    }

    return -1;
}

int main()
{
    int a[100], n, x, i, pos;

    printf("Enter number of sorted elements: ");
    scanf("%d", &n);

    printf("Enter sorted elements:\n");
    for (i = 0; i < n; i++)
        scanf("%d", &a[i]);

    printf("Enter value to search: ");
    scanf("%d", &x);

    pos = binarySearch(a, n, x);

    if (pos == -1)
        printf("Number is not present\n");
    else
        printf("Number is present at position %d\n", pos);

    return 0;
}
```

OR

Q 1) Perform the following operations using dynamic implementation of Queue

- i. Insert n elements in a queue.
- ii. Delete the element from the queue and display it.

Answer / C Program

```
#include <stdio.h>
#include <stdlib.h>

struct node
{
    int data;
    struct node *next;
};

struct node *front = NULL;
struct node *rear = NULL;

void insert(int x)
{
    struct node *newnode;

    newnode = (struct node *)malloc(sizeof(struct node));
    newnode->data = x;
    newnode->next = NULL;

    if (rear == NULL)
    {
        front = rear = newnode;
    }
    else
    {
        rear->next = newnode;
        rear = newnode;
    }
}

void delete()
{
    struct node *temp;

    if (front == NULL)
    {
        printf("Queue is empty\n");
        return;
    }

    temp = front;
    printf("Deleted element = %d\n", front->data);

    front = front->next;

    if (front == NULL)
        rear = NULL;

    free(temp);
}

void display()
{
    struct node *temp = front;
```

```

    printf("Queue: ");

    while (temp != NULL)
    {
        printf("%d ", temp->data);
        temp = temp->next;
    }

    printf("\n");
}

int main()
{
    int n, i, x;

    printf("Enter number of elements: ");
    scanf("%d", &n);

    for (i = 0; i < n; i++)
    {
        printf("Enter element: ");
        scanf("%d", &x);
        insert(x);
    }

    display();
    delete ();
    display();

    return 0;
}

```

Section II: Database Management Systems

[15 Marks]

Q 2) Create the following tables (primary keys are underlined).

Bus (Busno , capacity ,depot_name)

Route (Routeno ,source ,destination ,no_of_stations)

Relationship: Bus-Route : **Many to one**

Constraint: Bus capacity should not be null.

A. Create the above database in PostGreSQL and insert sufficient records.

Run line-by-line in psql:

```

CREATE DATABASE slip13_db;
\c slip13_db
DROP TABLE IF EXISTS bus, route CASCADE;

CREATE TABLE route(
    routeno int PRIMARY KEY,
    source varchar(50),
    destination varchar(50),
    no_of_stations int
);

CREATE TABLE bus(
    busno int PRIMARY KEY,
    capacity int NOT NULL,
    depot_name varchar(50),
    routeno int REFERENCES route(routeno)

```

```
);

INSERT INTO route VALUES
(41, 'Pune station', 'Swargate', 12), (42, 'Pimpri', 'Hadapsar', 15),
(43, 'Pune station', 'Kothrud', 2), (44, 'Swargate', 'Hadapsar', 8);

INSERT INTO bus VALUES
(201, 40, 'Swargate', 41), (202, 35, 'Pimpri', 42),
(203, 30, 'Swargate', 43), (204, 45, 'Swargate', 44);

SELECT * FROM route;
SELECT * FROM bus;
```

B. Execute the following queries in PostGreSQL (any 3)

i. List all buses at depot “Swargate”.

```
SELECT * FROM bus WHERE depot_name='Swargate';
```

ii. Delete Bus whose Bus number is “204”.

```
DELETE FROM bus WHERE busno=204;
```

iii. List all buses on route number 41.

```
SELECT * FROM bus WHERE routeno=41;
```

iv. List the route details having number of stations greater than 10.

```
SELECT * FROM route WHERE no_of_stations>10;
```

v. List all routes starting from “Pune station”.

```
SELECT * FROM route WHERE source='Pune station';
```

C. Write a query to delete all the buses on routes in which total stations are less than 3.

```
DELETE FROM bus WHERE routeno IN (SELECT routeno FROM route WHERE
no_of_stations < 3);
```

OR

C. Create a view to list all buses and route details from “Pimpri” to “Hadapsar”.

```
CREATE VIEW pimpri_hadapsar_buses AS SELECT
b.busno, b.capacity, b.depot_name, r.routeno, r.source, r.destination, r.no_of_stations
FROM bus b JOIN route r ON b.routeno=r.routeno WHERE r.source='Pimpri'
AND r.destination='Hadapsar';
```

Savitribai Phule Pune University

S.Y.B.Sc.(C.S.) NEP – 2020 Practical Examination

Lab Course CS-203-MJ-P (Lab Course based on CS-201 MJ-T & CS-202-MJ-T)

Duration: 3 Hours

Maximum Marks: 35

Section I: Data Structures

[15 Marks]

Q 1) Sort a random array of n integers (accept the value of n from user) in ascending order by using insertion sort algorithm.

OR

Q 1) Perform the following operations using dynamic implementation of Queue

- i. Insert n elements in a queue.
- ii. Delete the element from the queue and display it.

Section II: Database Management Systems

[15 Marks]

Q 2) Create the following tables (**primary keys are underlined**).

Game (gname, noofplayers, coachname, captain_name)

Player (pno, pname)

There exists a **one-to-many** relationship between Game and Player.

Constraint **captain_name** should be **uppercase**.

A. Create the above database in PostgreSQL and insert sufficient records. [5 marks]

B. Execute the following queries in PostgreSQL (any 3) [6 marks]

- i. List the names of players playing “hockey”.
- ii. Give the average number of players.
- iii. Delete the records of players playing ‘kho kho’.
- iv. List names of players not playing “cricket”.
- v. Update the coach name from “Dhoni” to “Tendulkar” for game “Cricket”.

C. Write a query to display the game details with maximum number of players. [4 marks]

OR

C. Create a view to print game-wise list of Players along with their game name

Q 3) Viva

[5 marks]

SLIP 15 — Solutions (Code & Queries)

Section I: Data Structures

[15 Marks]

Q 1) Sort a random array of n integers (accept the value of n from user) in ascending order by using insertion sort algorithm.

Answer / C Program

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

int main()
{
    int a[100], n, i, j, temp;

    printf("Enter number of elements: ");
    scanf("%d", &n);

    srand(time(0));

    printf("Random array: ");
    for (i = 0; i < n; i++)
    {
        a[i] = rand() % 100;
        printf("%d ", a[i]);
    }

    for (i = 1; i < n; i++)
    {
        temp = a[i];
        j = i - 1;

        while (j >= 0 && a[j] > temp)
        {
            a[j + 1] = a[j];
            j--;
        }

        a[j + 1] = temp;
    }

    printf("\nSorted array: ");
    for (i = 0; i < n; i++)
        printf("%d ", a[i]);

    return 0;
}
```

OR

Q 1) Perform the following operations using dynamic implementation of Queue

- i. Insert n elements in a queue.
- ii. Delete the element from the queue and display it.

Answer / C Program

```
#include <stdio.h>
#include <stdlib.h>

struct node
{
    int data;
    struct node *next;
};

struct node *front = NULL;
struct node *rear = NULL;

void insert(int x)
{
    struct node *newnode;

    newnode = (struct node *)malloc(sizeof(struct node));
    newnode->data = x;
    newnode->next = NULL;

    if (rear == NULL)
    {
        front = rear = newnode;
    }
    else
    {
        rear->next = newnode;
        rear = newnode;
    }
}

void delete()
{
    struct node *temp;

    if (front == NULL)
    {
        printf("Queue is empty\n");
        return;
    }

    temp = front;
    printf("Deleted element = %d\n", front->data);

    front = front->next;

    if (front == NULL)
        rear = NULL;

    free(temp);
}

void display()
{
    struct node *temp = front;
```

```

    printf("Queue: ");

    while (temp != NULL)
    {
        printf("%d ", temp->data);
        temp = temp->next;
    }

    printf("\n");
}

int main()
{
    int n, i, x;

    printf("Enter number of elements: ");
    scanf("%d", &n);

    for (i = 0; i < n; i++)
    {
        printf("Enter element: ");
        scanf("%d", &x);
        insert(x);
    }

    display();
    delete ();
    display();

    return 0;
}

```

Section II: Database Management Systems

[15 Marks]

Q 2) Create the following tables (primary keys are underlined).

Game (gname, noofplayers, coachname, captain_name)

Player (pno, pname)

There exists a **one-to-many** relationship between Game and Player.

Constraint **captain_name** should be **uppercase**.

A. Create the above database in PostGreSQL and insert sufficient records.

Run line-by-line in psql:

```

CREATE DATABASE slip15_db;
\c slip15_db
DROP TABLE IF EXISTS player, game CASCADE;

CREATE TABLE game(
    gname varchar(50) PRIMARY KEY,
    noofplayers int,
    coachname varchar(50),
    captain_name varchar(50) CHECK (captain_name = UPPER(captain_name))
);

CREATE TABLE player(
    pno int PRIMARY KEY,
    pname varchar(50),
    gname varchar(50) REFERENCES game(gname)
);

```

```

INSERT INTO game VALUES
('Hockey',11,'Coach1','ROHIT'),('Cricket',11,'Dhoni','VIRAT'),
('Kho Kho',9,'Coach3','AMIT'),('Football',11,'Coach4','RAHUL');

INSERT INTO player VALUES
(1,'Amit','Hockey'),(2,'Riya','Hockey'),(3,'Sahil','Cricket'),
(4,'Pooja','Cricket'),(5,'Kiran','Kho Kho'),(6,'Rohan','Football');

SELECT * FROM game;
SELECT * FROM player;

```

B. Execute the following queries in PostGreSQL (any 3)

i. List the names of players playing “hockey”.

```

SELECT p.pname FROM player p JOIN game g ON p.gname=g.gname WHERE
LOWER(g.gname)='hockey';

```

ii. Give the average number of players.

```

SELECT AVG(noofplayers) AS average_players FROM game;

```

iii. Delete the records of players playing ‘kho kho’.

```

DELETE FROM player WHERE gname='Kho Kho';

```

iv. List names of players not playing “cricket”.

```

SELECT p.pname FROM player p WHERE LOWER(p.gname)<>'cricket';

```

v. Update the coach name from “Dhoni” to “Tendulkar” for game “Cricket”.

```

UPDATE game SET coachname='Tendulkar' WHERE gname='Cricket' AND
coachname='Dhoni';

```

C. Write a query to display the game details with maximum number of players.

```

SELECT * FROM game WHERE noofplayers=(SELECT MAX(noofplayers) FROM game);

```

OR

C. Create a view to print game-wise list of Players along with their game name

```

CREATE VIEW game_players AS SELECT g.gname,p.pname FROM game g JOIN player p
ON g.gname=p.gname;

```

Savitribai Phule Pune University

S.Y.B.Sc.(C.S.) NEP – 2020 Practical Examination

Lab Course CS-203-MJ-P (Lab Course based on CS-201 MJ-T & CS-202-MJ-T)

Duration: 3 Hours

Maximum Marks: 35

Section I: Data Structures

[15 Marks]

Q 1) Read the data from the file “employee.txt” and sort on age using Bubble sort.

OR

Q 1) Perform the following operations using dynamic implementation of Queue

- i. Insert n elements in a queue.
- ii. Delete the element from the queue and display updated queue.

Section II: Database Management Systems

[15 Marks]

Q 2) Create the following tables (**primary keys are underlined**).

Order(orderno, s_order_date, amount)

Client(cno, name, address)

A client can give **one or more** orders, but an order belongs to exactly one client.

Constraint: Order amount should be > 0

A. Create the above database in PostGreSQL and insert sufficient records. [5 marks]

B. Execute the following queries in PostGreSQL (any 3) [6 marks]

- i. Change order date of client number '04' to '12/4/19'.
- ii. Display all orders given in the month of October.
- iii. Display all orders in descending order of amount.
- iv. Delete all the clients of Mumbai.
- v. Count the number of orders having amount greater than 40000.

C. Write a query to display the client name who has placed maximum number of orders. [4 marks]

OR

C. Create a view to list names of clients having more than two sales orders.

Q 3) Viva [5 marks]

SLIP 19 — Solutions (Code & Queries)

Section I: Data Structures

[15 Marks]

Q 1) Read the data from the file “employee.txt” and sort on age using Bubble sort.

Answer / C Program

```
#include <stdio.h>
#include <string.h>

struct employee
{
    char name[30];
    int age;
};

int main()
{
    FILE *fp, *out;
    struct employee e[100], temp;
    int n = 0, i, j;

    fp = fopen("employee.txt", "r");

    if (fp == NULL)
    {
        printf("File not found\n");
        return 0;
    }

    while (fscanf(fp, "%s %d", e[n].name, &e[n].age) == 2)
        n++;

    fclose(fp);

    for (i = 0; i < n - 1; i++)
    {
        for (j = 0; j < n - i - 1; j++)
        {
            if (e[j].age > e[j + 1].age)
            {
                temp = e[j];
                e[j] = e[j + 1];
                e[j + 1] = temp;
            }
        }
    }

    out = fopen("sortedemponage.txt", "w");

    for (i = 0; i < n; i++)
        fprintf(out, "%s %d\n", e[i].name, e[i].age);

    fclose(out);

    printf("Data sorted by age successfully.\n");
    printf("Sorted data written to sortedemponage.txt\n");

    return 0;
}
```

OR

Q 1) Perform the following operations using dynamic implementation of Queue

- i. Insert n elements in a queue.
- ii. Delete the element from the queue and display updated queue.

Answer / C Program

```
#include <stdio.h>
#include <stdlib.h>

struct node
{
    int data;
    struct node *next;
};

struct node *front = NULL;
struct node *rear = NULL;

void insert(int x)
{
    struct node *newnode;

    newnode = (struct node *)malloc(sizeof(struct node));
    newnode->data = x;
    newnode->next = NULL;

    if (rear == NULL)
    {
        front = rear = newnode;
    }
    else
    {
        rear->next = newnode;
        rear = newnode;
    }
}

void delete()
{
    struct node *temp;

    if (front == NULL)
    {
        printf("Queue is empty\n");
        return;
    }

    temp = front;
    printf("Deleted element = %d\n", front->data);

    front = front->next;

    if (front == NULL)
        rear = NULL;

    free(temp);
}

void display()
```

```

{
    struct node *temp = front;

    printf("Queue: ");

    while (temp != NULL)
    {
        printf("%d ", temp->data);
        temp = temp->next;
    }

    printf("\n");
}

int main()
{
    int n, i, x;

    printf("Enter number of elements: ");
    scanf("%d", &n);

    for (i = 0; i < n; i++)
    {
        printf("Enter element: ");
        scanf("%d", &x);
        insert(x);
    }

    display();
    delete ();
    display();

    return 0;
}

```

Section II: Database Management Systems

[15 Marks]

Q 2) Create the following tables (**primary keys are underlined**).

Order(orderno, s_order_date, amount)

Client(cno, name, address)

A client can give **one or more** orders, but an order belongs to exactly one client.

Constraint: Order amount should be > 0

A. Create the above database in PostGreSQL and insert sufficient records.

Run line-by-line in psql:

```

CREATE DATABASE slip19_db;
\c slip19_db
DROP TABLE IF EXISTS orders, client CASCADE;

CREATE TABLE client(
    cno int PRIMARY KEY,
    name varchar(50),
    address varchar(50)
);

CREATE TABLE orders(
    orderno int PRIMARY KEY,
    s_order_date date,
    amount numeric(12,2) CHECK (amount > 0),

```

```

    cno int REFERENCES client(cno) ON DELETE CASCADE
);

INSERT INTO client VALUES
(1, 'Amit', 'Pune'), (2, 'Riya', 'Mumbai'), (3, 'Sahil', 'Mumbai'), (4, 'Pooja', 'Nashik');

INSERT INTO orders VALUES
(101, '2019-04-10', 25000, 1), (102, '2019-10-05', 45000, 1),
(103, '2019-10-15', 55000, 2), (104, '2019-08-20', 30000, 3), (105, '2019-10-
25', 70000, 2);

SELECT * FROM client;
SELECT * FROM orders;

```

B. Execute the following queries in PostgreSQL (any 3)

i. Change order date of client number '04' to '12/4/19'.

```
UPDATE orders SET s_order_date=DATE '2019-04-12' WHERE cno=4;
```

ii. Display all orders given in the month of October.

```
SELECT * FROM orders WHERE EXTRACT(MONTH FROM s_order_date)=10;
```

iii. Display all orders in descending order of amount.

```
SELECT * FROM orders ORDER BY amount DESC;
```

iv. Delete all the clients of Mumbai.

```
DELETE FROM client WHERE address='Mumbai';
```

v. Count the number of orders having amount greater than 40000.

```
SELECT COUNT(*) FROM orders WHERE amount>40000;
```

C. Write a query to display the client name who has placed maximum number of orders.

```
SELECT c.name, COUNT(o.orderno) AS order_count FROM client c JOIN orders o ON
c.cno=o.cno GROUP BY c.cno, c.name ORDER BY order_count DESC LIMIT 1;
```

OR

C. Create a view to list names of clients having more than two sales orders.

```
CREATE VIEW clients_more_than_2_orders AS SELECT c.name, COUNT(o.orderno) AS
order_count FROM client c JOIN orders o ON c.cno=o.cno GROUP BY c.cno, c.name
HAVING COUNT(o.orderno)>2;
```